

Tree Recursion

Announcements

Converting Iteration to Recursion (Continued)

Twenty-One Rules

Two players alternate turns, on which they can add 1, 2, or 3 to the current total

The total starts at 0

The game end whenever the total is 21 or more

The last player to add to the total loses

Discussion Question: Play Twenty-One

Rewrite play as a recursive function without a while statement.

- Do you need to define a new inner function? Why or why not? If so, what are its arguments?
- What is the base case and what is returned for the base case?

```
def play(strategy0, strategy1, goal=21):  
    """Play twenty-one and return the winner.
```

```
>>> play(some_strat, some_other_strat)
```

```
1  
"""
```

```
n = 0
```

```
who = 0 # Player 0 goes first
```

```
while n < goal:
```

```
    if who == 0:
```

```
        n = n + strategy0(n)
```

```
        who = 1
```

```
    elif who == 1:
```

```
        n = n + strategy1(n)
```

```
        who = 0
```

```
return who
```

```
def play(strategy0, strategy1, goal=21):  
    """Play twenty-one and return the winner.
```

```
>>> play(some_strat, some_other_strat)
```

```
1  
"""
```

```
def f(n, who):
```

```
    if n >= goal:
```

```
        return who
```

```
    if who == 0:
```

```
        n = n + strategy0(n)
```

```
        who = 1
```

```
    elif who == 1:
```

```
        n = n + strategy1(n)
```

```
        who = 0
```

```
    return f(n, who)
```

```
return f(0, 0)
```

Recursion Tips

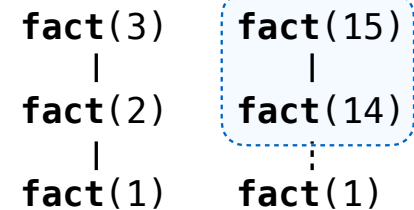
Effective Recursion

How to Know That a Recursive Case is Implemented Correctly?

```
def fact(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n * fact(n-1)
```

Tracing: Diagram the whole computational process (only feasible for very small examples)

Induction: Check that $f(n)$ is correct as long as $f(n-1) \dots f(0)$ are.
(*This the recursive leap of faith.*) (**Functional Abstraction!**)



How to Get Started on a Recursive Function?

Start with the recursive case: Come up with a process that involves recursion by writing down calls you might make, what they would return, and how to use those return values.

Base case(s): Once you have the recursive case, you can trace to see where it leads.

What else helps?

Convert from iteration: If the function is easy for you to write using **while**, then you could convert it later to recursion.

Describe the recursion: Start with a description of the recursive case before writing code.

Spring 2024 Midterm 1 Question 4(e)

Definition. A *dice integer* is a positive integer whose digits are all from 1 to 6.

```
def streak(n):  
    """Return whether positive n is a dice integer in which all the digits are the same.  
  
    >>> streak(22222)  
    True  
    >>> streak(4)  
    True  
    >>> streak(22322)  # 2 and 3 are different digits.  
    False  
    >>> streak(99999)  # 9 is not allowed in a dice integer.  
    False  
    """  
    return (n >= 1 and n <= 6) or (n > 9 and n % 10 == n // 10 % 10 and streak(n // 10))
```

Come up with a process: how can we use the result of streak on a smaller input?

Consider example recursive calls you might make: streak(2222), streak(2), ...

A Process: In a streak, every digit except the last is a streak, and the last matches

Mutual Recursion

Mutually Recursive Functions

Two functions `f` and `g` are mutually recursive if `f` calls `g` and `g` calls `f`.

```
def unique_prime_factors(n):  
    """Return the number of unique prime factors of n.
```

```
def smallest_factor(n):  
    "The smallest divisor of n above 1."
```

```
>>> unique_prime_factors(51) # 3 * 17
```

```
2
```

```
>>> unique_prime_factors(9) # 3 * 3
```

```
1
```

```
>>> unique_prime_factors(72000) = 2 * 2 * 2 * 2 * 2 * 2 * 3 * 3 * 5 * 5 * 5
```

```
3
```

```
.....
```

```
k = smallest_factor(n)
```

```
def no_k(n):
```

```
    "Return the number of unique prime factors of n other than k."
```

```
    if n == 1:
```

```
        return 0
```

```
    elif n % k != 0:
```

```
        return unique_prime_factors(n)
```

```
    else:
```

```
        return no_k(n // k)
```

```
    return 1 + no_k(n)
```

Find the smallest
(prime) factor

Keep removing that factor until
only other factors are left

And then count the
prime factors of that

count	what remains
0	72000
1 (k = 2)	1125
2 (k = 3)	125
3 (k = 5)	1

Tree Recursion

Counting Partitions

The number of partitions of a positive integer n , using parts up to size m , is the number of ways in which n can be expressed as the sum of positive integer parts up to m in increasing order.

`count_partitions(6, 4)`

~~$1 + 5 = 6$~~ $5 > m (4)$

~~$4 + 2 = 6$~~ Decreasing order

$2 + 4 = 6$

$1 + 1 + 4 = 6$

$3 + 3 = 6$

$1 + 2 + 3 = 6$

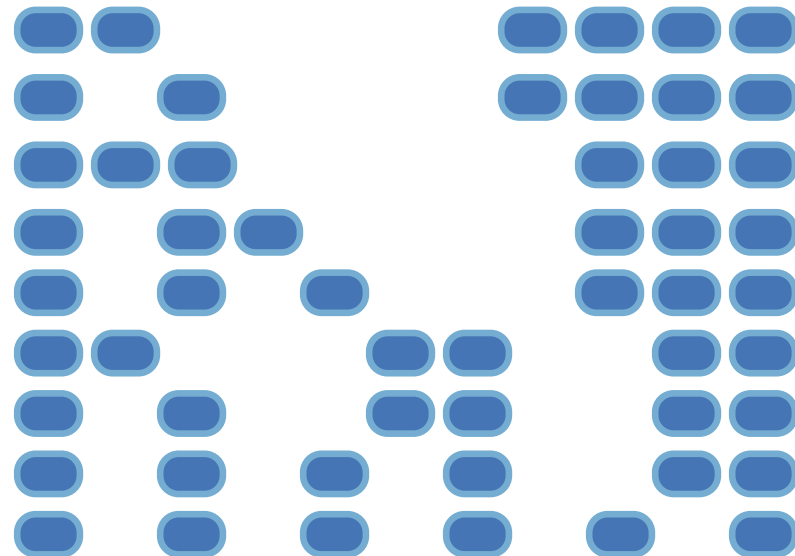
$1 + 1 + 1 + 3 = 6$

$2 + 2 + 2 = 6$

$1 + 1 + 2 + 2 = 6$

$1 + 1 + 1 + 1 + 2 = 6$

$1 + 1 + 1 + 1 + 1 + 1 = 6$

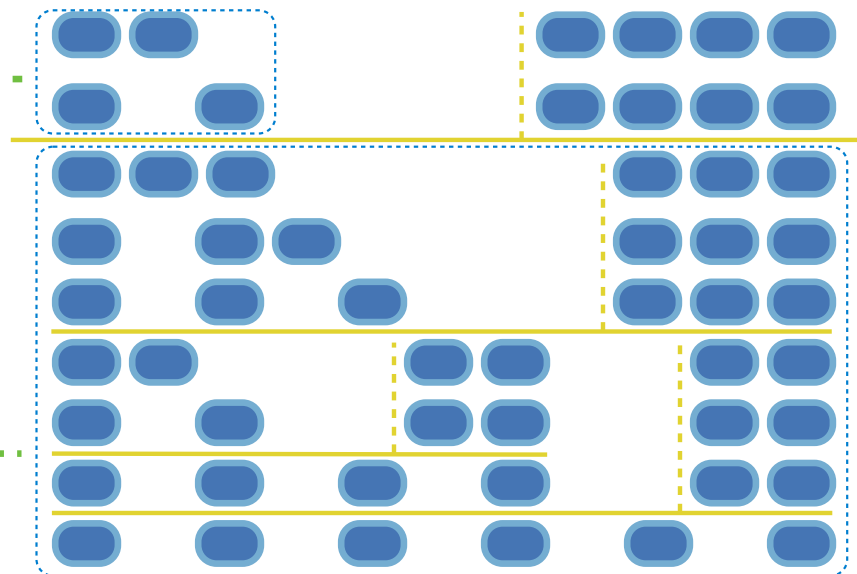


Counting Partitions

The number of partitions of a positive integer n , using parts up to size m , is the number of ways in which n can be expressed as the sum of positive integer parts up to m in non-decreasing order.

`count_partitions(6, 4)`

- Recursive decomposition: finding simpler instances of the problem.
- Explore two possibilities:
 - Use at least one 4
 - Don't use any 4
- Solve two simpler problems:
 - `count_partitions(2, 4)`
 - `count_partitions(6, 3)`
- Tree recursion often involves exploring different choices.



Counting Partitions

The number of partitions of a positive integer n , using parts up to size m , is the number of ways in which n can be expressed as the sum of positive integer parts up to m in increasing order.

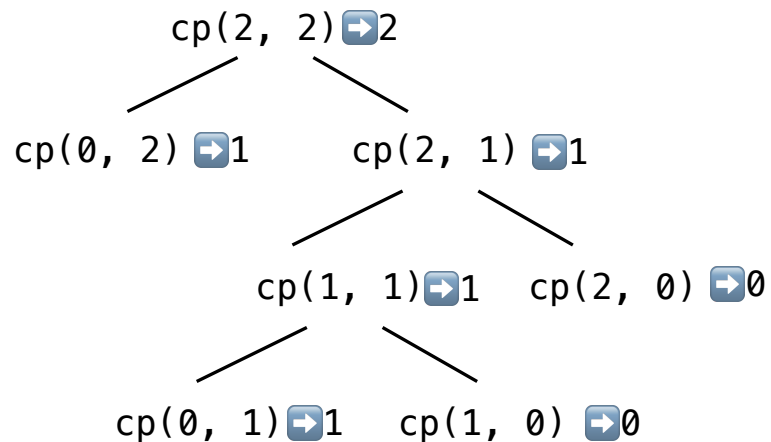
- Recursive decomposition: finding simpler instances of the problem.
- Explore two possibilities:
 - Use at least one 4
 - Don't use any 4
- Solve two simpler problems:
 - `count_partitions(2, 4)` -----
 - `count_partitions(6, 3)` -----
- Tree recursion often involves exploring different choices.

```
def count_partitions(n, m):
    if n == 0:
        return 1 # We found a way
    elif n < 0:
        return 0 # We subtracted too much
    elif m == 0:
        return 0 # We ran out of numbers
    else:
        with_m = count_partitions(n-m, m)
        without_m = count_partitions(n, m-1)
        return with_m + without_m
```

(Demo)

Tracing Count Partitions

The evaluation of `count_partitions(2, 2)` is tree structured.



```
def count_partitions(n, m):  
    if n == 0:  
        return 1 # We found a way  
    elif n < 0:  
        return 0 # We subtracted too much  
    elif m == 0:  
        return 0 # We ran out of numbers  
    else:  
        with_m = count_partitions(n-m, m)  
        without_m = count_partitions(n, m-1)  
        return with_m + without_m
```

Spring 2023 Midterm 2 Question 5

Definition. When parking vehicles in a row, a motorcycle takes up 1 parking spot and a car takes up 2 adjacent parking spots. A string of length n can represent n adjacent parking spots using % for a motorcycle, <> for a car, and . for an empty spot.

For example: '%.<><>' (Thanks to the Berkeley Math Circle for introducing this question.)

Implement **count_park**, which returns the number of ways that vehicles can be parked in n adjacent parking spots for positive integer n . Some or all spots can be empty.

```
def count_park(n):
    """Count the ways to park cars and motorcycles in n adjacent spots.
    >>> count_park(1) # '.' or '%'
    2
    >>> count_park(2) # '.. ', '%.', '%.', '%%', or '<>'
    5
    >>> count_park(4) # some examples: '<><>', '%.%.', '%<>%', '%.<>'
    29
    """
    if n < 0:
        return 0
    elif n == 0:
        return 1
    else:
        return count_park(n-2) + count_park(n-1) + count_park(n-1)
```